

数据库系统应试背诵精简版
第一部分：全考点背诵版（优先级排序）
一、关系代数（必考，五种基本运算）
选择 σ 条件(r): 取满足条件的行 投影 π 列(r): 取指定列，去重 并 $r \cup s$: 合并，去重（要求同元性） 差 $r - s$: 在 r 不在 s 笛卡尔积 $r \times s$: 所有元组组合 更名 ρ 新名(r)
附加运算（可由基本导出）:
- 交 $r \cap s = r - (r - s)$
- 自然连接 $r \bowtie s$: 相同属性等值连接后去重复列
- 外连接: 左 $\Join$ 、右 $\Join$ 、全 $\Join$ （保留未匹配，填 null）
- 聚集 $g_{\sigma}f(r)$: 分组聚集（avg,min,max,sum,count）
最大工资查询（复合示例）:
π salary(instructor) - π instructor.salary(σ instructor.salary < d.salary (instructor $\times$ ρ_d (instructor)))
二、sql 数据定义语言 (ddl)
create table 表名 (列名 数据类型 [primary key foreign key references 表(列)], 列名 数据类型 [not null unique check(条件)]);
drop table 表名;
alter table 表名 add 列名 数据类型;
alter table 表名 drop column 列名;
三、sql 数据查询语言 (dql)
select [distinct] 列 1, 列 2, 聚集函数(列) as 别名 from 表 1 [join 表 2 on 条件] where 条件 group by 分组列 having 分组后条件 order by 排序列 [asc desc];
连接类型
-- 内连接
select * from a inner join b on a.key = b.key;
-- 左外连接
select * from a left outer join b on a.key = b.key;
-- 右外连接
select * from a right outer join b on a.key = b.key;
-- 全外连接
select * from a full outer join b on a.key = b.key;
-- 自然连接（同名等值，去重列）
select * from a natural join b;
-- using（指定同名属性）
select * from a join b using (共同列);
嵌套子查询
where 列 in (select ...);
where exists (select ...);
where 列 > any (select ...);
where 列 > all (select ...);
聚集与分组
select dept_name, avg(salary) from instructor group by dept_name;
select dept_name, avg(salary) from instructor group by dept_name having avg(salary) > 50000;
四、sql 数据操纵语言 (dml)
insert into 表名 (列 1,列 2) values (值 1,值 2);
insert into 表名 select ...;
update 表名 set 列 = 值 where 条件;
update 表名 set 列 = case when 条件 then 值 else 值 end;
delete from 表名 where 条件;
五、视图
create view 视图名 as select ...;
虚表，无实际数据，用于安全与逻辑独立性 仅行列子集视图可更新
六、完整性约束
实体完整性: primary key（非空且唯一） 参照完整性: foreign key references ... on delete cascade set null set default 用户定义: not null, unique, check(条件) 断言: create assertion 断言名 check(谓词);
七、授权
grant select, insert, update, delete on 表名 to 用户名 [with grant option];
revoke select on 表名 from 用户名 [cascade restrict];
授权图: 节点（用户+权限），边（授权关系），回收时级联
八、存储过程与函数
create function 函数名(参数 类型) returns 类型 begin ... return 值; end;
create procedure 过程名(参数 类型) begin ...; end; call 过程名(参数);
外部语言过程（java, c#）效率高

九、触发器	
create trigger 触发器名	
{before after} {insert update delete} on 表名	
[for each row]	
begin	
-- 触发动作	
end;	
十、e-r 模型与转换（必背）	
基本要素	
• 实体集（矩形），联系集（菱形），属性（椭圆） • 双线：全参与；单线：部分参与 • 弱实体集（双矩形）：无主码，主码 = 强实体主码 + 分辨符	
联系类型与转换规则	
联系类型	转换规则
1:1	任一方加对方主码作为外码
1:n	n 方加一方主码作为外码
m:n	新建关系，含双方主码+联系属性，主码为双方主码组合
弱实体	新建关系，属性为弱实体属性+强实体主码，主码为强实体主码+分辨符
多元联系	新建关系，含所有参与实体主码+联系属性，主码为所有主码组合
扩展 e-r	
• 特化/概化：子类继承父类属性；自顶向下（特化） vs 自底向上（概化） • 不相交特化（d） vs 重叠特化（o）；全概化（total） vs 部分概化（缺省） • 聚集：将联系集视为高层实体，用于联系上的联系	
十一、关系模式优化（范式与函数依赖）	
函数依赖（fd）	
• $x \rightarrow y$：若 $t1.x = t2.x$ 则 $t1.y = t2.y$ • armstrong 公理：自反律（$x \supseteq y \Rightarrow x \rightarrow y$）、增广律（$x \rightarrow y \Rightarrow xz \rightarrow yz$）、传递律（$x \rightarrow y, y \rightarrow z \Rightarrow x \rightarrow z$） • 导出规则：合并律（$x \rightarrow y, x \rightarrow z \Rightarrow x \rightarrow yz$）、分解律（$x \rightarrow yz \Rightarrow x \rightarrow y, x \rightarrow z$）、伪传递律（$x \rightarrow y, wy \rightarrow z \Rightarrow wx \rightarrow z$） • 属性集团包 x^+：由 x 通过 fd 能推出的所有属性集。用于判断 x 是否为超键（$x^+ = r$）或 fd 是否成立（$y \subseteq x^+$）	
范式	
• 1nf：属性原子性 • 2nf：1nf + 非主属性完全依赖于候选码（消除部分依赖） • bcnf：每个非平凡 fd $x \rightarrow a$，x 必须是超键 • 3nf：每个非平凡 fd $x \rightarrow a$，要么 x 是超键，要么 a 是主属性（即属于某个候选码）	
分解标准	
• 无损连接分解：r 分解为 $r1, r2$，满足 $r1 \cap r2 \rightarrow r1$ 或 $r1 \cap r2 \rightarrow r2$ • 保持函数依赖：分解后 fd 的并集等价于原 fd 集	
bcnf 分解算法	
对违反 bcnf 的 fd $x \rightarrow a$ ，分解为 (xUa) 和 $(r-a)$ ，重复。	
3nf 分解算法（保持依赖且无损）	
1. 计算正则覆盖 fc 2. 对 fc 中每个 $fd\ x \rightarrow y$，创建模式 (xUy) 3. 若没有模式包含 r 的候选键，则添加一个候选键模式	
十二、物理设计（存储与索引）	
文件组织	
• 定长记录 / 变长记录 • 堆文件、顺序文件、散列文件、多表聚集文件	
顺序索引	
• 主索引：索引顺序与数据顺序相同（仅一个） • 辅助索引：索引顺序与数据顺序不同（多个），必须是稠密索引 • 稠密索引：每个搜索码一个索引项 • 稀疏索引：部分搜索码有索引项（仅主索引可用） • 多级索引：在索引上建索引，减少 i/o	
b*树索引	
• 平衡树，所有叶节点深度相同 • 每个节点至少 $\lceil n/2 \rceil$ 个指针（根除外） • 查询、插入、删除效率 $O(\log_a n)$ • 叶节点顺序集支持范围查询	
散列索引	
• 静态散列：固定桶数，可能溢出 • 动态散列（可扩展散列）：桶数动态增长，使用散列前缀 i，桶地址表大小 2^i	

十三、查询处理与优化
查询处理步骤
1. 解析与语法检查 $\rightarrow$ 2. 查询优化（代数优化+物理优化） $\rightarrow$ 3. 执行
选择操作实现
全表扫描（小表） 索引扫描（等值或范围条件）
连接操作实现（按代价从高到低）

嵌套循环连接 排序-合并连接（先排序后合并） 索引连接（利用内表索引） hash 连接（适用于等值连接，较小表构建哈希表）
表达式计算
物化: 中间结果存为临时表 流水线: 边计算边传递，不存中间结果 查询重写）
代数优化
选择下移（尽早做选择） 投影下移（去掉无用列） 连接顺序优化（先连接小结果集）
十四、事务管理
事务特性（acid）
原子性: 全做或全不做 一致性: 事务前后数据库保持一致 隔离性: 并发事务互不干扰 持久性: 提交后永久保存
事务状态
活动 $\rightarrow$ 部分提交 $\rightarrow$ 提交 / 失败 $\rightarrow$ 中止
调度与可串行化
串行调度: 事务依次执行 可串行化调度: 并发调度结果等价于某个串行调度 冲突可串行化: 通过交换非冲突指令得到串行调度 优先图: 节点为事务，边 $ti \rightarrow tj$ 表示冲突且 ti 先于 tj, 有环则不可串行化（从低到高）
隔离级别
read uncommitted（读未提交） read committed（读已提交） repeatable read（可重复读） serializable（可串行化）
十五、并发控制技术
锁
共享锁（s 锁）: 读，可多个共存 排他锁（x 锁）: 写，互斥 锁相容矩阵: s 与 s 相容，其余皆不相容
两阶段封锁协议（2pl）
扩展阶段: 只加锁，不放锁 收缩阶段: 只放锁，不加锁 保证冲突可串行化 严格 2pl: 事务结束前不放 x 锁 $\rightarrow$ 避免级联回滚 强 2pl: 事务结束前不放任何锁
死锁
等待图: 有环则死锁 死锁解除: 回滚一个事务 死锁预防: 一次封锁法、顺序封锁法
多粒度锁
意向锁（is, ix, six）: 表示下层将要加 s/x 锁 锁粒度: 数据库、表、页、元组
基于时间戳的协议
每个事务分配时间戳 ts(ti) 每个数据项有 r-timestamp（最后读时间）和 w-timestamp（最后写时间） 读写规则: 若 $ts(ti) < w\text{-timestamp}(x)$ 则回滚（读太旧）; 若 $ts(ti) < r\text{-timestamp}(x)$ 则回滚（写太旧） 保证冲突可串行化，但可能不可恢复（需增加提交依赖）
快照隔离
事务看到启动时的已提交数据快照 写-写冲突时后者回滚
乐观并发控制（有效性检查）
三阶段: 读执行 $\rightarrow$ 验证 $\rightarrow$ 写 验证阶段检查与其他事务的读写集合是否有冲突
十六、备份与恢复
故障类型
事务故障、系统故障、介质故障
日志（wal, 预写日志）
每个事务有 $\langle t \text{ start} \rangle, \langle t, x, \text{old_value}, \text{new_value} \rangle, \langle t \text{ commit} \rangle, \langle t \text{ abort} \rangle$
恢复策略
redo: 重做已提交事务的更新 undo: 撤销未提交事务的更新 延迟修改: commit 前不写数据库（只需 redo） 立即修改: 可能同时需要 undo 和 redo
检查点
在日志中记录 $\langle \text{checkpoint} \rangle$（I 为活动事务列表） 恢复时从最后一个检查点开始，只对检查点之后的事务做 redo/undo
十七、dbms 体系结构
三级模式与两级映射
外模式（用户视图） $\rightarrow$ 概念模式（逻辑结构） $\rightarrow$ 内模式（物理存储） 逻辑独立性: 概念模式变，外模式不变 物理独立性: 内模式变，概念模式不变
系统结构
集中式、客户-服务器（c/s）、浏览器-服务器（b/s） 并行系统: 共享内存、共享磁盘、无共享 分布式数据库: 场地自治 + 全局透明

第二部分：必考大题题型与规范答题步骤

题型一：属性集闭包计算

知识点：给定函数依赖集 f 和属性集 x，求 x⁺；判断 x 是否为候选码；判断 fd 是否被蕴含。

解题步骤：

- result = x
 - 重复：对每个 fd (a→b)，若 a⊆result，则 result = result ∪ b 3. 直到 result 不再扩大
 - 若 result 包含所有属性，则 x 是超键；再检查是否有真子集也能推出全属性，若无则 x 是候选码。
- 例题 1：** r(a,b,c,g,h,i)，f = {a→b, a→c, cg→h, cg→i, b→h}。求(ag)⁺，并判断 ag 是否为候选码。

解：
- result = {a,g}
- 用 a→b: result={a,b,g}
- 用 a→c: result={a,b,c,g}
- 用 b→h: result={a,b,c,g,h}
- 用 cg→h: 已有，无变化
- 用 cg→i: result={a,b,c,g,h,i}
- 包含所有属性，ag 是超键。检查真子集：
a⁺={a,b,c,h} 不含 g,i; g⁺={g}。
故 ag 无冗余，是候选码。

题型二：最小依赖集（正则覆盖）

知识点：消除冗余属性、冗余依赖，得到最小依赖集 fc。

解题步骤：

- 分解右侧为单属性。
 - 对每个 fd，检查左侧冗余：去掉某属性后计算闭包，若仍能推出右侧则删除该属性。
 - 检查每个 fd 是否冗余：去掉该 fd 后计算左侧闭包，若能推出右侧则删除该 fd。
 - 合并相同左侧的 fd。
- 例题 2：** f={a→bc, b→c, a→b, ab→c}，求最小依赖集。
- 解：**
- 拆分右侧：a→b, a→c, b→c, a→b, ab→c → 去重得 a→b, a→c, b→c, ab→c。
- 检查 ab→c: 去掉 b，计算 a⁺={a,b,c} 包含 c，故 b 冗余，改为 a→c；已有 a→c，所以 ab→c 可删。
- 检查 a→c: 去掉它，剩余 a→b, b→c, a⁺={a,b,c} 包含 c，冗余，删 a→c。
- 剩余 a→b, b→c。无相同左侧。故 fc={a→b, b→c}。

题型三：候选码求解

知识点：给定 f，找出 r 的所有候选码。

解题步骤：

- 找出不出现在任何 fd 右侧的属性，它们必在候选码中，记为 x。
 - 计算 x⁺，若等于全属性，则 x 为唯一候选码。
 - 否则，尝试加入其他属性，计算闭包，找到所有候选码。 4. 注意对称 fd 可能导致多个候选码。
- 例题 3：** r(a,b,c,d)，f={a→b, b→c, c→a, d→b}，求候选码。
- 解：**
- 不在右侧的属性：d（只出现在左侧）。候选码必须含 d。 - d⁺ = {d,b,c,a} = 全属性，故 d 是候选码。
- 不含 d 的能否？ a⁺={a,b,c} 不含 d； b⁺={b,c,a} 不含 d； c⁺={c,a,b} 不含 d。
故无其他候选码。唯一候选码：d。

题型四：范式判断与分解

知识点：判断 2nf,3nf,bcnf；分解为 bcnf（无损）；分解为 3nf（无损+保持依赖）。

解题步骤（bcnf 分解）：

- 找出候选码。
 - 对每个非平凡 fd x→a，若 x 不是超键，则违反 bcnf。
 - 分解为 (xUa) 和 (r-a)，重复。
- 解题步骤（3nf 保持依赖分解）：**
- 计算正则覆盖 fc。
 - 对每个 fd x→y in fc，创建模式 (xUy)。
 - 若没有模式包含候选键，则添加一个候选键模式。
- 例题 4：** r(a,b,c,d)，f={a→b, b→c, c→d}。(1) 求候选码。(2) 判断范式。(3) bcnf 分解。(4) 3nf 保持依赖分解。

解：
(1) a⁺={a,b,c,d}，候选码 a。
(2) 非主属性 b,c,d。有 b→c, c→d 传递依赖，不满足 3nf，但满足 2nf（无部分依赖）。故属于 2nf。
(3) bcnf 分解：
- 违反 fd: b→c (b 不是超键)，分解为 r1(b,c)，r2(a,b,d)。
- r2 上 fd: a→b (a 是超键)，无其他非平凡 fd，故 r2 符合 bcnf。结果 r1(b,c)，r2(a,b,d)。无损 (r1∩r2={b}, b→c)，但不保持 c→d。
(4) 3nf 保持依赖分解：
- fc={a→b, b→c, c→d}（已最小）
- 建模式：r1(a,b)，r2(b,c)，r3(c,d)
- 候选码 a 已在 r1 中，结果：r1(a,b), r2(b,c), r3(c,d)（保持所有依赖，无损）。

题型五：关系代数表达式

知识点：用关系代数表示查询，包括除法、极值。

例题 5： 学生表 student(sno,sname)，选课表 sc(sno,cno,grade)，课程表 course(cno,cname)。写出：
(1) 选修了“数据库”课程的学生学号。(2) 选修了所有课程的学生学号。(3) 最高成绩。
解：
(1) π sno (σ cname='数据库' (student $\bowtie$ sc $\bowtie$

course))
(2) π sno (sc) $\div$ π cno (course)
(3) π grade (sc) $- \pi$ sc1.grade (σ sc1.grade < sc2.grade (ρ sc1(sc) $\times$ ρ sc2(sc)))

题型六：sql 语句

知识点：单表查询、多表连接、子查询、分组、视图、授权、更新删除。

例题 6： 表结构 student(sno,sname,age,dept)，course(cno,cname,credit)，sc(sno,cno,grade)。写出 sql：
(1) 计算机系年龄>20 的学生姓名。(2) 每个学生的平均成绩，按平均分降序。(3) 选修了所有课程的学生学号。(4) 删除没有选课的学生。(5) 创建视图 v_high_grade（成绩≥90）。(6) 授予 u1 select on student with grant option。
解：
-- (1)
select sname from student where dept = 'cs' and age > 20;
-- (2)
select sno, avg(grade) as avg_grade from sc group by sno order by avg_grade desc;
-- (3)
select sno from student s where not exists (
select * from course c where not exists (
select * from sc where sc.sno = s.sno and
sc.cno = c.cno
)
);
-- (4)
delete from student where sno not in (select distinct sno from sc);
-- (5)
create view v_high_grade as
select sc.sno, course.cname, sc.grade
from sc, course
where sc.cno = course.cno and sc.grade >= 90;
-- (6)
grant select on student to u1 with grant option;

题型七：c-r 图设计与关系模式转换

知识点：实体、属性、联系、映射基数，转换为关系模式并标注主外码。

例题 7： 部门（部门号、部门名、电话），员工（工号、姓名、工资）。每个员工属于一个部门，部门有多个员工。部门与员工存在 1:1 的“管理”联系（一个部门有一个经理，一个员工只能管理一个部门）。画 c-r 图并转换。
解：
- 实体：部门（部门号，部门名，电话），员工（工号，姓名，工资）
- 联系 1：属于（1:n）
- 联系 2：管理（1:1）
- 转换：
部门(部门号,部门名,电话,经理工号) 主码:部门号
外码:经理工号 参照 员工(工号)
员工(工号,姓名,工资,部门号) 主码:工号 外码:部门号 参照 部门(部门号)

题型八：查询树与代数优化

知识点：将 sql 转换为语法树，应用选择下移、投影下移优化。

例题 8： 查询“计算机系工资>50000 的教师姓名及所授课程”，表 instructor(id,name,dept,salary)，teaches(id,course_id)。写出优化前后代数表达式及查询树。
解：
- 优化前： π name,course_id (σ dept='cs' $\wedge$ salary>50000 $\wedge$ instructor.id=teaches.id (instructor $\times$ teaches))
- 优化后： π name,course_id ((σ dept='cs' $\wedge$ salary>50000 (instructor)) $\bowtie$ teaches)

题型九：事务调度与可串行化（优先图）

知识点：冲突操作、优先图、冲突可串行化判断。

例题 9： 调度如下，判断是否冲突可串行化。

时间	t1	t2
1	r(a)	
2		w(a)
3	w(a)	
4		r(a)

解：
- 冲突对：t1 r(a) 与 t2 w(a) → t1→t2; t2 w(a) 与 t1 w(a) → t2→t1。有环，不可串行化。

题型十：两阶段封锁与死锁

知识点：2pl 协议、死锁检测与预防。

例题 10： 事务 t1 和 t2，
t1: lock-x(a); read(a); a=a-10; write(a); lock-s(b); read(b); unlock(b); unlock(a);
t2: lock-s(a); read(a); unlock(a); lock-x(b); read(b); b=b+20; write(b); unlock(b)。
问：调度是否 2pl？是否可能死锁？
解：
- t1: 先 lock-x(a), lock-s(b)后 unlock(a)再 unlock(b) — 所有加锁在第一次解锁前，符合 2pl。
- t2: lock-s(a)后 unlock(a)，再 lock-x(b) — 加锁与解锁交替，违反 2pl。
- 死锁可能：若 t2 改为先 lock-x(b)后 lock-s(a)，则 t1 lock-x(a)成功，t2 lock-x(b)成功，然后相互请求对方锁形成死锁。

题型十一：b*树插入与删除

知识点：b*树结构、插入分裂、删除合并。

例题 11： b*树阶数 n=4（每个节点最多 3 键，最少 2 键，根除外）。依次插入 2,5,8,12,15,20，画出最后树。

解：

- 插入 2,5,8 → 根[2,5,8]
- 插入 12 → 叶满分裂：左[2,5]，右[8,12]，根[8]
- 插入 15 → 右叶[8,12,15]
- 插入 20 → 右叶[8,12,15,20]满，分裂为[8,12]和[15,20]，中键 15 上提，根[8,15]，叶：左[2,5]，中[8,12]，右[15,20]

题型十二：基于日志的恢复

知识点：日志格式、检查点、redo/undo。

例题 12： 日志如下，系统崩溃，如何恢复？
<t0 start>
<t0, a, 100, 200>
<t1 start>
<checkpoint {t0,t1}>
<t0, b, 200, 300>
<t0 commit>
<t1, c, 50, 80>
解：
- 从检查点向后扫描：t0 已提交 → redo（a=200,b=300），t1 未提交 → undo（c=50）。最终 a=200,b=300,c=50。

题型十三：无损连接与保持依赖判定

知识点：两模式无损判定（交→一方）；多模式用表格法；保持依赖检查投影。

例题 13： r(a,b,c,d)，f={a→b, b→c, c→d}，分解 ρ ={r1(a,b,c), r2(c,d)}。判断无损和保持依赖。
解：
- 无损：r1∩r2={c}，在 r2 中 c→d 成立，c→r2，故无损。
- 保持依赖：r1 上 fd 有 a→b,b→c；r2 上有 c→d。原 f 中 a→b,b→c,c→d 均能检查，故保持。

资料总结：王亚东